

install.sh

Revision: 2 **Last modified:** 2026-06-29T00:00:00Z **Authority:** vasic-digital tmux project **Maintainer:** milosvasic **Scope:** §11.4.18 script companion doc for the curl-obtainable one-shot installer (source: scripts/install.sh)

Overview

scripts/install.sh is the modern-CLI-style installer for the vasic-digital optimized + verified hardened tmux build. It is obtained and triggered with a single curl command, exactly like rustup, nvm, or brew's bootstrap:

```
curl -fsSL https://raw.githubusercontent.com/vasic-digital/tmux/main/scripts/install.sh | bash
```

It is **fully self-contained** — it makes no assumption that the repository already exists on the host, so it runs correctly when piped straight from curl into bash. In one invocation it:

1. **Preflights** — confirms git is present and resolves the install root by the constitution naming convention (§11.4.29 lowercase snake_case): the default is \$HOME/tmux (the project directory name is tmux).
2. **Clones recursively** — git clone --recurse-submodules of the whole project plus a belt-and-suspenders git submodule update --init --recursive, so constitution/, tmux/, and Containers/ are all populated.
3. **Builds + verifies + installs** — delegates to scripts/setup.sh, which builds the binary for the host OS, runs the verification gate, and — only on a GREEN gate (§11.4) — installs ~/.tmux.conf and appends the PATH+session snippet to the host's shell rc.
4. **Validates** — runs the full scripts/tests/run_all.sh suite and surfaces the PASS/FAIL/SKIP summary; a FAIL makes the installer exit non-zero.

5. **Confirms PATH wiring** — reports which shell rc (~/.bashrc / ~/.zshrc) carries the snippet and whether tmx is resolvable, then tells you to source the rc (or open a new terminal).

The bar is honest: any failure in clone / build / verify / test makes the installer exit non-zero — never a silent green (§11.4 anti-bluff covenant).

Prerequisites

- **git** — required (the only hard dependency of `install.sh` itself).
- A **C toolchain** (`build_native.sh`: a compiler + `libevent-dev` + `libncurses-dev`, plus `autoconf/automake/pkg-config/bison`) **OR** a **container engine** (podman/docker) for the hermetic build. `scripts/setup.sh` chooses the right path for the host; missing build deps surface as a non-zero exit with guidance, never a fake success.
 - Linux build deps: `sudo bash scripts/install_deps.sh` (one-time).
 - macOS: `brew install podman jemalloc` (or the native toolchain).
- Missing runtime deps (e.g. **jemalloc**) are obtained git-ignored into `.local-deps/` automatically by `scripts/obtain_local_deps.sh` during setup (§11.4.77). See `obtain_local_deps.md`.
- **No sudo is run by install.sh**. Under a `curl | bash` pipe it will not auto-escalate (§12 host-session safety); if host build deps are missing, run the documented `install_deps.sh` step yourself first.

Usage

```
# Canonical one-liner (clones to $HOME/tmux, builds, verifies,
# tests, wires PATH)
curl -fsSL https://raw.githubusercontent.com/vasic-digital/tmux/
main/scripts/install.sh | bash

# Pass options under the pipe (bash reads the script from stdin →
# options after '-s --')
curl -fsSL <raw-url>/scripts/install.sh | bash -s -- --dir ~/work/
tmux

# Or download then run
curl -fsSL <raw-url>/scripts/install.sh -o install.sh && bash
install.sh
```

Environment overrides (env OR flag — flag wins)

Env var	Flag	Default	Meaning
TMX_INSTALL_DIR	--dir DIR	\$HOME/tmux	install root (§11.4.29 naming)
TMX_REPO_URL	--repo URL	git@github.com:vasic- digital/tmux.git	clone source (SSH / git protocol)
TMX_INSTALL_BRANCH	-- branch B	main	branch to clone / track
TMX_INSTALL_NO_SETUP=1	-- clone- only	(off)	stop after clone+submodules (no build, no host writes)
TMX_INSTALL_DETECT_RC_ONLY=1	-- detect- rc-only	(off)	print the shell rc PATH would be wired into, then exit
TMX_INSTALL_HTTPS_REWRITE=1	-- https- rewrite	(off)	opt in to the git@github: → https:// github.com/ submodule URL rewrite (keyless aid; see the warning below)
TMX_INSTALL_NO_HTTPS_REWRITE=1	--no- https- rewrite	(off)	legacy explicit opt-OUT; still honoured, and wins over the opt- in

SSH (the git protocol) is the default clone scheme, and the submodule URL rewrite is OFF by default (changed in 1.0.44). The installer clones over the git@github.com: URLs exactly as .gitmodules pins them, so your SSH key authenticates every fetch.

Why the previous HTTPS-by-default was removed (TMX-086): the rewrite converted the **private** nested submodule constitution/submodules/helix_perf_cache (git@github.com:HelixDevelopment/helix_perf_cache.git) into an *unauthenticated* HTTPS fetch. GitHub answers that by asking for a username, and because nothing set

`GIT_TERMINAL_PROMPT=0` the installer BLOCKED on that prompt forever — with no way to answer it under `curl | bash`. Measured A/B on the same repo, same host, same minute: SSH `exit=0`; SSH+rewrite `exit=128` could not read Username.

The installer now also exports `GIT_TERMINAL_PROMPT=0` (plus empty `GIT_ASKPASS/SSH_ASKPASS`), so a missing credential can only ever produce a fast, readable error — never a silent hang. `GIT_SSH_COMMAND` defaults to `ssh -o StrictHostKeyChecking=accept-new -o ConnectTimeout=20` so a first-contact github host key does not block either; an ssh key **passphrase** prompt is deliberately still allowed, and your own `GIT_SSH_COMMAND` is never overridden.

Edge cases

- **Install dir already is our checkout** → the installer **updates** it (`git fetch + git pull --ff-only + recursive submodule update`) instead of re-cloning. Idempotent: re-running the installer is safe.
- **Install dir exists, is non-empty, and is NOT our repo** → the installer **refuses to clobber** it (§9.2 absolute data safety) and exits non-zero, telling you to pick another `TMX_INSTALL_DIR` or remove it yourself.
- **No network** → the canonical HTTPS clone needs network. The installer can also clone from a local mirror (`TMX_REPO_URL=/path/to/mirror` or a `file:///... URL`); for a local mirror it enables `protocol.file.allow` so the submodule clones resolve locally. This is the same machinery the test harness uses to validate the installer entirely offline.
- **Private submodule with no access** → an HTTPS rewrite does NOT grant access to a private submodule. The installer surfaces the submodule failure honestly (the constitution/ presence check warns rather than faking green, §11.4.6); provide an SSH key or an HTTPS credential/token for the private repo.
- **SSH-keyed user needs a PRIVATE submodule** → this is the default path since 1.0.44 and needs no flag: the pinned `git@github.com:` URLs are used as-is and your key authenticates. Do **not** set `TMX_INSTALL_HTTPS_REWRITE=1` — that rewrites the submodule's SSH URL to an **unauthenticated** HTTPS URL and the private fetch then fails (historically: hung on a credential prompt, TMX-086). See [../guides/troubleshooting.md §4](#).
- **Keyless user with no SSH key at all** → set `TMX_INSTALL_HTTPS_REWRITE=1` **and** pass an HTTPS clone source, e.g. `--repo https://github.com/vasic-digital/tmux.git`. Honest

boundary (§11.4.6): this reaches the public repos only. The private nested submodule constitution/submodules/helix_perf_cache still cannot be fetched without credentials, so the install will fail on it — with a fast, readable error rather than a hang. A keyless install of the full tree is not currently possible; an SSH key with access is required.

- **Build deps missing / verification RED** → setup.sh exits non-zero and the installer surfaces it (never PATH-exports an unverified binary, §11.4).
- **Validation suite FAILs** → the installer prints the SUMMARY and exits non-zero (honest, never a silent green).

Internal behaviour

```
phase 1  preflight      : require git; resolve install root; decide
clone|update;
                                §9.2 refuse-to-clobber a foreign non-empty
dir
phase 2  obtain         : git clone --recurse-submodules  (or
fetch+pull --ff-only    on update) + git submodule update --init --
recursive;
                                assert constitution/Constitution.md landed
    └─ TMX_INSTALL_NO_SETUP=1 stops here (clone-only seam)
phase 3  setup.sh       : build + verify gate + (GREEN-gated) host
config install
phase 4  run_all.sh     : full validation suite; FAIL → installer
exits non-zero
phase 5  PATH confirm:  report which rc carries the snippet + tmx
resolvability
phase 6  summary        : next-step (source ~/.bashrc or ~/.zshrc),
usage
```

Two test seams keep the installer validatable without mutating the host: `TMX_INSTALL_NO_SETUP=1` (stop after the clone) and `TMX_INSTALL_DETECT_RC_ONLY=1` (print the rc-detection result and the naming-convention default, no clone). The `git -c` config flags (`protocol.file.allow, url....insteadOf`) propagate to the recursive submodule subprocesses via `GIT_CONFIG_PARAMETERS`.

File exports installed (what ends up on the host)

After a GREEN `setup.sh`, the operator's host carries:

- The `PATH`+session snippet appended to the host's shell rc — `~/.bashrc` AND/OR `~/.zshrc` (and, for bash login shells, `~/.bash_profile` / `~/.profile`).
- `~/.tmux.conf` — the generated tmux configuration (a non-ours pre-existing `~/.tmux.conf` is backed up to `~/.tmux.conf.pre-vasic-digital`).
- `scripts/tmx` (generated wrapper) prepended onto `PATH` so `tmx` resolves to this project's verified build; the system `tmux` stays reachable side-by-side.

Related scripts

- `setup.sh` — the build + verify + install pipeline the installer delegates to (§11.4 GREEN-gated `PATH` export).
- `obtain_local_deps.md` / `scripts/obtain_local_deps.sh` — §11.4.77 git-ignored local-dependency obtain/resolve mechanism.
- `uninstall.md` / `scripts/uninstall.sh` — removal (delegates to `setup.sh --uninstall`).
- `scripts/tests/run_all.sh` — the full validation suite the installer runs.
- `scripts/tests/69_install_script.sh` — the anti-bluff test for this installer (offline recursive-clone proof + rc-detection unit + §9.2 refuse + idempotency).

Last verified

2026-06-29 — `bash -n scripts/install.sh clean; scripts/tests/69_install_script.sh PASS=9 FAIL=0 SKIP=0` across 3 consecutive runs (deterministic, §11.4.50/§11.4.98); real offline recursive clone landed `constitution/Constitution.md` + `scripts/setup.sh` + `scripts/tmx.template`; §9.2 refuse-to-clobber proven (exit 4, foreign file intact); idempotent re-run entered update mode.

Sources verified 2026-06-29

- `git submodule / recursive clone semantics + protocol.file.allow` (CVE-2022-39253 hardening) and `GIT_CONFIG_PARAMETERS` propagation — `git help submodule`, `git help config` (local git version 2.50.1).
- `url.<base>.insteadOf` for keyless HTTPS fetch of public submodules — `git help config (url.insteadOf)`.